

Tema 4.2. Herencia

1. En orientación a objetos, ¿qué es la **herencia** y su relación con "A es-un B"? Explica las dos implicaciones principales: (1) **compatibilidad de tipos** y (2) **herencia de estado y comportamiento**. Pon un ejemplo en Java muy sencillo, donde un **Soldado** tiene un **nombre** (privado) y un método **saludar()** que muestra su nombre. Hay dos subtipos: un **Artillero**, que es capaz de disparar cohetes y un **Zapador** que pone minas, ambos heredan el atributo nombre y la capacidad de saludar. Además, y de forma específica, el artillero tiene un número de cohetes y el zapador un número de minas, accesibles mediante "getters" específicos. Respecto a la compatibilidad de tipos, aprovechémosla: crea un array de **Soldado**, mete varios de distinto tipo (son todos compatibles con **Soldado**). Recórrela y que todos te saluden.

Respuesta

La herencia en orientación a objetos es un mecanismo por el cual una clase (subclase) puede reutilizar y extender el comportamiento de otra (superclase). Se expresa con la relación "A es-un B". Por ejemplo, un Artillero es-un Soldado. Esto implica dos ideas clave: primero, que existe compatibilidad de tipos (un objeto de la subclase puede tratarse como si fuera de la superclase), y segundo, que la subclase hereda estado y comportamiento (atributos y métodos) de la superclase.

La compatibilidad de tipos permite escribir código más general. Si se tiene un array de Soldado, se pueden guardar dentro objetos de Artillero y Zapador. Por otro lado, la herencia de estado y comportamiento evita duplicar código: todos los soldados tienen nombre y pueden saludar, sin necesidad de redefinirlo en cada clase.

```
class Soldado { private final String nombre;
```

```
    public Soldado(String nombre) {
        this.nombre = nombre;
    }

    public void saludar() {
        System.out.println("Soy " + nombre);
    }
}
```

```
class Artillero extends Soldado { private final int cohetes;
```

```
    public Artillero(String nombre, int cohetes) {
        super(nombre);
        this.cohetes = cohetes;
    }
}
```

```
}

public int getCohetes() {
    return cohetes;
}
```

```
}
```

```
class Zapador extends Soldado { private final int minas;
```

```
    public Zapador(String nombre, int minas) {
        super(nombre);
        this.minas = minas;
    }

    public int getMinas() {
        return minas;
    }
```

```
}
```

```
public class Main { public static void main(String[] args) { Soldado[] ejercito = new Soldado[3];
```

```
    ejercito[0] = new Artillero("Juan", 5);
    ejercito[1] = new Zapador("Luis", 3);
    ejercito[2] = new Artillero("Ana", 7);

    for (Soldado s : ejercito) {
        s.saludar();
    }
}
```

```
}
```

2. Al crear los soldados concretos, ¿cuántos constructores se ejecutan y en qué orden? ¿Qué significa **super** dentro de un constructor? Si la clase base no tiene visible el constructor sin parámetros, ¿debo llamar a **super** siempre?

Respuesta

Al crear un objeto de una subclase, se ejecutan tantos constructores como niveles haya en la jerarquía. Primero se ejecuta el constructor de la superclase y después el de la subclase. Esto ocurre porque la parte "Soldado" del objeto debe inicializarse antes que la parte específica del Artillero o Zapador.

La palabra clave **super** se usa dentro del constructor de la subclase para llamar al constructor de la superclase. Esta llamada debe ser la primera instrucción del constructor. Sirve para inicializar correctamente los atributos

heredados, especialmente si son privados.

Si la clase base no tiene un constructor sin parámetros accesible, es obligatorio llamar a super explícitamente indicando los parámetros necesarios. En caso contrario, el código no compila, porque Java no puede inicializar automáticamente la superclase.

3. Respecto a los objetos de subclases en memoria, los atributos privados de la superclase, ¿forman parte de una instancia de la subclase en memoria? En caso afirmativo ¿implica que se puedan usar desde el código de la subclase? Explícalo con el ejemplo de **Soldado** y alguna de sus subclases.

Respuesta

Sí, los atributos privados de la superclase forman parte de la instancia de la subclase en memoria. Es decir, un objeto Artillero contiene internamente toda la información de un Soldado, incluyendo su nombre.

Sin embargo, el hecho de que estén en memoria no significa que se puedan acceder directamente desde la subclase. Si un atributo es private, solo es accesible desde la propia clase donde se declara.

Por ejemplo, en Soldado, el atributo nombre no puede usarse directamente en Artillero. Para acceder a él, sería necesario un método público o protegido en Soldado, como un getter.

4. ¿Qué implica en términos de **extensibilidad** de código el hecho de que sean compatibles a nivel de tipos? Ilustra esto añadiendo un nuevo tipo de **Soldado** y demostrando que el código para pedir el saludo a todos los soldados no se modifica.

Respuesta

La compatibilidad de tipos permite que el código sea fácilmente extensible. Si se añade una nueva subclase de Soldado, no es necesario modificar el código que ya trabaja con Soldado.

Por ejemplo, se puede añadir un nuevo tipo:

```
class Medico extends Soldado { public Medico(String nombre) { super(nombre); }
```

```
    public void curar() {  
        System.out.println("Curando...");  
    }  
}
```

El código que recorre el array de Soldado no necesita cambiar. Simplemente se puede añadir un Medico al array, y seguirá funcionando correctamente porque también es un Soldado.

Esto demuestra que la herencia permite ampliar el sistema sin modificar código existente, lo que mejora el mantenimiento y la escalabilidad.

5. En Java, cuando trabajo con referencias y herencia. ¿Puedo tener una referencia del supertipo que apunte a objetos reales de un subtipo? ¿Puedo invocar con la referencia del supertipo a métodos públicos del subtipo? ¿En qué consiste el **"upcasting"** y el **"downcasting"**? ¿Qué es el **instanceof**? Pon un ejemplo de recorrido de un array de **Soldado**, comprobando que, si el objeto real es un **Artillero**, solicite el número de cohetes que tiene y los imprima.

Respuesta

Sí, en Java se puede tener una referencia del supertipo que apunte a un objeto real de un subtipo. Esto es precisamente lo que permite la herencia y se conoce como upcasting. Por ejemplo, un Soldado puede referenciar a un Artillero.

Sin embargo, con una referencia del supertipo solo se pueden invocar métodos definidos en la superclase. Para acceder a métodos específicos del subtipo, es necesario hacer un downcasting, que consiste en convertir la referencia al tipo concreto.

El operador instanceof permite comprobar en tiempo de ejecución si un objeto es de un tipo concreto antes de hacer el casting.

```
for (Soldado s : ejercito) { s.saludar();
```

```
    if (s instanceof Artillero) {  
        Artillero a = (Artillero) s;  
        System.out.println("Cohetes: " + a.getCohetes());  
    }  
}
```

Esto evita errores en tiempo de ejecución al asegurar que el objeto realmente es del tipo esperado.

6. Respecto a la ocultación de información y herencia, ¿qué significa acceso **"protegido"** de métodos y/o atributos? ¿Cómo se implementa en Java? Pon un ejemplo de uso de en la clase **Soldado** para que su nombre sea protegido y pueda usarse en el método de poner bombas del **Zapador**.

Respuesta

El acceso protegido (protected) permite que un atributo o método sea accesible dentro de su propia clase y también desde sus subclases. Es un nivel intermedio entre private y public.

Se implementa en Java usando la palabra clave protected. Esto es útil cuando se quiere permitir que las subclases utilicen directamente ciertos atributos sin hacerlos completamente públicos.

```
class Soldado { protected final String nombre;
```

```
public Soldado(String nombre) {  
    this.nombre = nombre;  
}
```

```
}
```

```
class Zapador extends Soldado { public Zapador(String nombre, int minas) { super(nombre); }
```

```
public void ponerMina() {  
    System.out.println(nombre + " está colocando una mina");  
}
```

```
}
```

En este caso, Zapador puede usar directamente nombre gracias al acceso protegido.

7. En los lenguajes orientados a objetos ¿hay una **clase base** para todos los objetos? ¿Ocurre en todos los lenguajes? ¿Qué ocurre en Java?

Respuesta

En muchos lenguajes orientados a objetos existe una clase base común para todos los objetos. Esta clase actúa como raíz de la jerarquía de herencia.

En Java, esta clase es Object. Todas las clases heredan directa o indirectamente de Object, incluso si no se especifica explícitamente.

Esto permite que todos los objetos compartan ciertos métodos básicos como toString(), equals() o hashCode(). Sin embargo, no todos los lenguajes orientados a objetos tienen esta característica de forma obligatoria.

8. ¿Qué es la "**herencia múltiple**"? ¿Existe en Java herencia múltiple?

Respuesta

La herencia múltiple consiste en que una clase puede heredar de más de una clase base al mismo tiempo. Esto permite combinar comportamientos de varias clases.

Sin embargo, puede generar problemas como ambigüedades (por ejemplo, si dos clases base tienen un método con el mismo nombre). Este problema se conoce como el "problema del diamante".

En Java, no existe herencia múltiple de clases. Solo se puede heredar de una única clase. Sin embargo, Java permite implementar múltiples interfaces, lo que ofrece una forma controlada de reutilizar comportamiento sin los problemas de la herencia múltiple.

9. Las excepciones en los lenguajes orientados a objetos son objetos. Por tanto, se pueden crear excepciones personalizadas. Pon un ejemplo en Java de una excepción personalizada

(**UsuarioNoEncontradoException**), que sea *no controlada* y que además este compuesto con un **Usuario**, para saber qué **Usuario** dio el problema. Permite además que se pueda incluir la causa, es decir, sobrecarga el constructor para tener una versión que permita añadir la causa subyacente.

Respuesta

Las excepciones en Java son clases que heredan de `Exception` o `RuntimeException`. Para crear una excepción no controlada, se debe extender `RuntimeException`.

Además, se puede componer la excepción con otros objetos, como un `Usuario`, para aportar más información sobre el error.

```
class Usuario { private final String nombre;
```

```
    public Usuario(String nombre) {  
        this.nombre = nombre;  
    }  
  
    public String getNombre() {  
        return nombre;  
    }  
}
```

```
class UsuarioNoEncontradoException extends RuntimeException { private final Usuario usuario;
```

```
    public UsuarioNoEncontradoException(Usuario usuario) {  
        super("Usuario no encontrado: " + usuario.getNombre());  
        this.usuario = usuario;  
    }  
  
    public UsuarioNoEncontradoException(Usuario usuario, Throwable causa) {  
        super("Usuario no encontrado: " + usuario.getNombre(), causa);  
        this.usuario = usuario;  
    }  
  
    public Usuario getUsuario() {  
        return usuario;  
    }  
}
```

De este modo, la excepción no solo indica el error, sino también el usuario implicado y, opcionalmente, la causa del problema.

10. Herencia vs. Composición. Se dice que no se debe emplear herencia simplemente por reutilizar código, es decir, que si quiero reutilizar código simplemente, no debo pensar en herencia como primera opción ¿por qué?

Respuesta

No se debe usar herencia únicamente para reutilizar código porque implica una relación fuerte entre clases. Cuando una clase hereda de otra, queda acoplada a su implementación.

Esto puede provocar problemas si la superclase cambia, ya que puede afectar a todas las subclases. Además, no siempre existe una relación "es-un", que es la base conceptual de la herencia.

Si solo se busca reutilizar código, suele ser mejor emplear composición, ya que permite usar funcionalidades de otra clase sin crear una dependencia tan fuerte.

11. Herencia vs. Composición. Se dice que se debe "*favorecer la composición frente a la herencia*", ¿por qué?

Respuesta

Se recomienda favorecer la composición frente a la herencia porque la composición es más flexible. Permite cambiar el comportamiento en tiempo de ejecución y combinar objetos de distintas formas.

En cambio, la herencia es más rígida, ya que la relación entre clases queda fijada en tiempo de compilación. Además, las jerarquías profundas de herencia suelen ser difíciles de mantener.

La composición permite construir objetos a partir de otros sin necesidad de encajar en una jerarquía estricta, lo que facilita el diseño modular y reutilizable.

12. Herencia vs. Composición. Se dice que la "*herencia rompe la encapsulación*", ¿a qué se refiere esto?

Respuesta

Se dice que la herencia rompe la encapsulación porque las subclases dependen de detalles internos de la superclase. Esto hace que cambios en la implementación interna puedan afectar a las subclases.

Por ejemplo, si una subclase usa atributos protegidos o depende de cómo funcionan ciertos métodos internos, cualquier cambio en la superclase puede provocar errores.

Esto reduce la capacidad de modificar la superclase sin afectar al resto del sistema, lo que va en contra del principio de ocultación de información.

13. Pongamos un ejemplo de dos alternativas para lo mismo. Tenemos un **Estudiante** y un **Trabajador**, ambos tienen datos en común: el DNI y el nombre. Modelemos esto de dos formas: uno por herencia, con una superclase **Persona**, y otro con composición, con una clase **DatosPersonales**. Se debe recibir una instancia de **DatosPersonales** en el constructor de la clase **Estudiante** y **Trabajador**.

Respuesta

Una forma de modelar la situación es usando herencia, creando una clase base Persona que contiene los atributos comunes, y clases Estudiante y Trabajador que heredan de ella.

```
class Persona { private final String dni; private final String nombre;
```

```
    public Persona(String dni, String nombre) {  
        this.dni = dni;  
        this.nombre = nombre;  
    }  
}
```

```
class Estudiante extends Persona { public Estudiante(String dni, String nombre) { super(dni, nombre); } }  
class Trabajador extends Persona { public Trabajador(String dni, String nombre) { super(dni, nombre); } }
```

Otra forma es mediante composición, creando una clase DatosPersonales y usándola dentro de Estudiante y Trabajador.

```
class DatosPersonales { private final String dni; private final String nombre;
```

```
    public DatosPersonales(String dni, String nombre) {  
        this.dni = dni;  
        this.nombre = nombre;  
    }  
}
```

```
class Estudiante { private final DatosPersonales datos;  
}
```

```
    public Estudiante(DatosPersonales datos) {  
        this.datos = datos;  
    }  
}
```

```
class Trabajador { private final DatosPersonales datos;  
}
```

```
    public Trabajador(DatosPersonales datos) {  
        this.datos = datos;  
    }  
}
```

La primera opción usa herencia para expresar "es-un", mientras que la segunda usa composición para expresar "tiene-un".